705.641.81: Natural Language Processing Self-Supervised Models

Assignment 5: RNNs and Transformers/Self-Attention

For homework deadline and collaboration policy, please see our Canvas page.
Name: Brian W. Onieal
Collaborators, if any: None
Sources used for your homework, if any: Claude (Anthropic), as described in the GenAI disclosure below. Course slides for Modules 4, 5, and 6. A. Vaswani et al., "Attention Is All You Need," NeurIPS 2017. R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," ICML 2013 (question 7, cited on the Module 4 slides). A. Radford et al., "Improving Language Understanding by Generative Pre-Training," OpenAI, 2018 (question 2). N. Carlini et al., "Stealing Part of a Production Language Model," ICML 2024, and OpenAI Help Center, "Using logit bias to alter token probability with the OpenAI API," read October 19, 2026 (Section 2). S. Wang, B. Z. Li, M. Khabsa, H. Fang, and H. Ma, "Linformer: Self-Attention with Linear Complexity," arXiv:2006.04768, 2020 (Sections 4.3 and 4.4). The Assignment 5 skeleton code and the PyTorch documentation linked in the assignment.

> GenAI disclosure: Claude (Anthropic, model claude-opus-5-5) wrote the answers and derivations in this submission and filled in the code TODOs. It also wrote the helper and verification scripts and ran the training on my laptop's GPU (NVIDIA RTX 4050 Laptop, 6 GB) through the Claude app. I chose the scope and reviewed the work before submitting. Where an answer says "I tested" or "I timed," it describes a run Claude did on my laptop at my direction. Each `.py` file Claude wrote or changed says the same at its top. The other skeleton files are unchanged.
>
> Code: https://github.com/brianonieal/NLP-Assignment-5

This assignment focuses on basic understanding of Recurrent Neural Networks and Transformers for language modeling. We will also touch upon Tokenization.

Homework goals: After completing this homework, you should be comfortable with:

- aligning your prior understanding of Neural Net training for RNNs;
- thinking about Self-Attention module and implementing a simple Transformer language model.

1 Concepts, intuitions and big picture

Each question might have multiple correct answers. (Check all that apply).

1. Which of these types of models would you use for completing prompts with generated text?

☐ An encoder model

☒ A decoder model

> Answer: a decoder model. A decoder is trained to predict the next token from the tokens to its left. That gives a distribution over the next token, which I can sample from, append to the prompt, and repeat. An encoder sees the whole input in both directions and learns to fill in masked tokens. It has no next-token distribution to generate from. The Module 6 slides say decoders like GPT-2 are "nice to generate from." The GPT I built in Section 5 is decoder only, and the completions at the end of this write-up came from that loop.

2. Which of these types of models would you use for classifying text inputs according to certain labels?

☒ An encoder model

☐ A decoder model

> Answer: an encoder model. An encoder builds each token's representation from the words on both sides of it, so the whole input is in view before anything gets labeled. A small classification head on top maps that to the labels. For BERT it's a linear layer on the `[CLS]` vector. The Module 6 slides list BERT and RoBERTa as encoders that capture bidirectional context. A decoder can be fine-tuned to classify too, and GPT-1 did it with a linear layer on the last token's state (Radford et al., 2018). Each position in a decoder only sees what's to its left, though. The encoder is the one I'd pick, and it's the only box I checked.

3. To which of these tasks would you apply a many-to-one RNN architecture?

☐ Speech recognition (input an audio clip and output a transcript)

☒ Sentiment classification (input a piece of text and output a 0/1 to denote positive or negative sentiment)

☒ Gender recognition from speech (input an audio clip and output a label indicating the speaker's gender)

> Answer: sentiment classification and gender recognition from speech. Many-to-one means a whole sequence goes in and one output comes out at the end. Sentiment classification reads every word of the text and outputs a single 0 or 1. Gender recognition reads every frame of the audio clip and outputs one label. Speech recognition outputs a sequence of words for a sequence of audio frames, so it's many-to-many.

4. What possible source can the bias observed in a model have?

☒ The model is a fine-tuned version of a pretrained model and it picked up its bias from it.

☒ The data the model was trained on is biased.

☒ The metric the model was optimizing for is biased.

> Answer: all three. A fine-tuned model starts from the pretrained weights. Whatever bias pretraining put there carries over unless fine-tuning removes it, and a small fine-tuning

set usually won't. Biased training data teaches the bias directly, since the model learns the patterns in its data. The metric matters too. The model optimizes whatever metric I pick and nothing else. For example, plain accuracy on data where one group is rare can reward a model that ignores that group.

5. How many dimensions does the tensor output by a decoder Transformer model have, and what are they?

☐ 2: The sequence length and the batch size

☐ 2: The sequence length and the hidden size

☒ 3: The sequence length, the batch size, and the hidden size

Answer: 3, the sequence length, the batch size, and the hidden size. A decoder outputs one hidden vector per token for every sequence in the batch. In my Assignment 5 GPT, `CausalSelfAttention.forward` unpacks this shape as `B, T, C = x.size()`. With gpt-nano that's (batch size, up to 32, 48). PyTorch puts the batch first, but the three dimensions are the ones in the third option. The language model head then maps the last dimension to the 50,257-token vocabulary, and the result is still 3-dimensional.

6. You are training an RNN language model. At the tth time step, what is the RNN doing? Choose the best answer.

☐ Estimating P($y_1, y_2, \ldots, y_{t-1}$)

☐ Estimating P(y_1)

☒ Estimating P($y_t | y_1, y_2, \ldots, y_{t-1}$)

☐ Estimating P($y_t | y_1, y_2, \ldots, y_t$)

Answer: the third option, estimating $P(y_t \mid y_1, \ldots, y_{t-1})$. By step t the RNN has read $y_1, \ldots, y_{t-1}$, and its hidden state summarizes them. The softmax output is a distribution over the word at t given those words, and training scores it with cross-entropy against the real y_t. Multiplying these conditionals over t gives the probability of the whole sentence by the chain rule. The fourth option conditions on y_t itself, which would hand the model the answer.

7. You are training an RNN, and find that your weights and activations are all taking on the value of `NaN` ("Not a Number"). Which of these is the most likely cause of this problem?

☐ Vanishing gradient problem.

☒ Exploding gradient problem.

☐ ReLU activation function g(.) used to compute g(z), where z is too large.

☐ Sigmoid activation function g(.) used to compute g(z), where z is too large.

Answer: exploding gradient problem. Backprop through time multiplies by $\mathbf{W}_{hh}^{\top}$ once for every step it goes back. Section 3.2.2 below shows the term $(\mathbf{W}_{hh}^{\top})^{t-i}$. If the largest eigenvalue of $\mathbf{W}_{hh}$ is above 1 in size, that product grows exponentially with $t - i$ (Module 4 slides 81 and 82). The gradient overflows to inf, the update pushes the weights to inf, and inf minus inf gives NaN, which then spreads to every activation. Vanishing gradients shrink toward 0 and don't produce NaN. ReLU on a large z just returns z. Sigmoid on a

large z saturates at 1 and its gradient goes to 0, which is the vanishing case. The fix on the Module 4 slides is gradient clipping (Pascanu et al., 2013). The Assignment 5 trainer does it with `grad_norm_clip = 1.0`.

8. You're done training an RNN language model. You're usig it to sample random sentences as follows. What are you doing at each time step t?

☐ (i) Use the probabilities output by the RNN to pick the highest probability word for that time-step as y_t. (ii) Then pass the ground-truth word from the training set to the next time-step.

☐ (i) Use the probabilities output by the RNN to randomly sample a chosen word for that time-step as y_t. (ii) Then pass the ground-truth word from the training set to the next time-step.

☐ (i) Use the probabilities output by the RNN to pick the highest probability word for that time-step as y_t. (ii) Then pass this selected word to the next time-step.

☒ (i) Use the probabilities output by the RNN to randomly sample a chosen word for that time-step as y_t. (ii) Then pass this selected word to the next time-step.

Answer: the fourth option. Sample y_t at random from the RNN's output distribution, then feed that sampled word in as the next input. Picking the highest-probability word is greedy decoding. It gives the same sentence every time and tends to repeat itself (Module 4 slide 96). Feeding the ground-truth word is teacher forcing, which only happens in training. After training there's no ground truth to feed. The Assignment 5 `generate` function does the fourth option with `do_sample=True`. It samples from the 40 most likely tokens and appends the result to the input.

9. You have a pet crab whose mood is heavily dependent on the current and past few days' weather. You've collected data for the past 42 days on the weather, which you represent as a sequence as $x_1, \ldots, x_{42}$. You've also collected data on your crab's mood, which you represent as $y_1, \ldots, y_{42}$. You'd like to build a model to map from $x \rightarrow y$. Should you use a Unidirectional RNN or Bidirectional RNN for this problem?

☐ Bidirectional RNN, because this allows the prediction of mood on day t to consider more information.

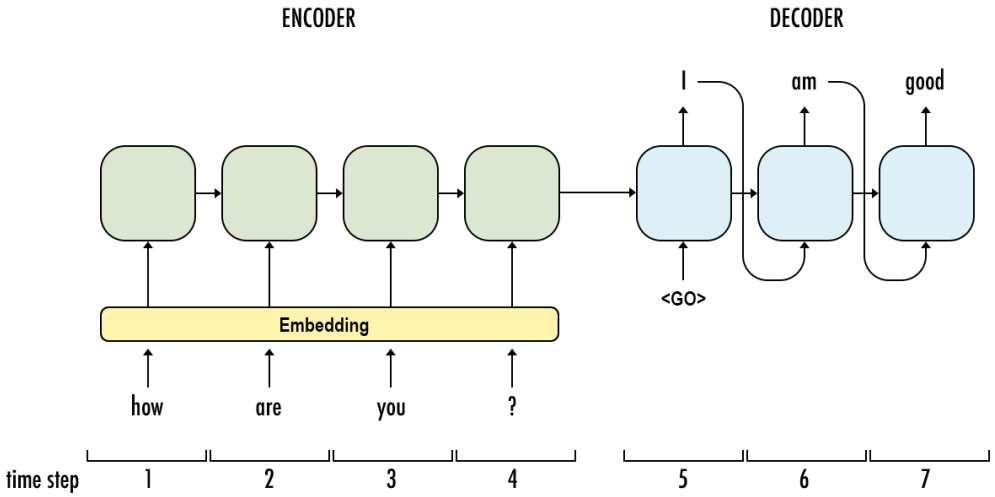
☐ Bidirectional RNN, because this allows backpropagation to compute more accurate gradients.

☒ Unidirectional RNN, because the value of y_t depends only on $x_1, \ldots, x_t$ but not on $x_{t+1}, \ldots, x_{42}$

☐ Unidirectional RNN, because the value of y_t depends only on x, and not other days' weather.

Answer: the third option, a unidirectional RNN, because y_t depends only on $x_1, \ldots, x_t$ and not on $x_{t+1}, \ldots, x_{42}$. The crab's mood depends on today's weather and the past few days'. A forward RNN's state at day t already holds days 1 through t. A bidirectional RNN would also read the weather after day t. That weather can't affect the mood on day t, and it wouldn't be known yet if I predicted day by day. The fourth option is wrong because the past few days' weather matters too.

10. Consider this encoder-decoder model.



This model is a "conditional language model" in the sense that the encoder portion (shown in green) is modeling the probability of the input sentence x.

☐ True ☒ False

> Answer: False. The encoder reads x and turns it into hidden states. Its last state starts the decoder, the same setup as the seq2seq slides in Module 4. It isn't trained to predict the words of x, so it doesn't model $P(x)$. The decoder is the language model. It models $P(y \mid x) = \prod_t P\,(y_t \mid y_{<t}, x)$, and conditioning on x through the encoder is what makes it a conditional language model.

11. Compared to the RNN-LMs and fixed-window-LMs, we expect the attention-based LMs to have the greatest advantage when:

☒ The input sequence length is large.

☐ The input sequence length is small.

> Answer: the input sequence length is large. Attention connects any two positions in one step. In an RNN, information from i steps back has to pass through i updates of the hidden state. The gradient has to pass back through them too, and that's where it vanishes. A fixed-window LM can't see anything outside its window at all. With a short input all three can see what they need, so attention's advantage mostly goes away. The Module 5 slides say self-attention is "better at maintaining long-distance dependencies" and needs $O(1)$ sequential operations instead of $O(n)$.

12. What is a model head?

☐ A component of the base Transformer network that redirects tensors to their correct layers

☐ Also known as the self-attention mechanism, it adapts the representation of a token according to the other tokens of the sequence

☒ An additional component, usually made up of one or a few layers, to convert the transformer predictions to a task-specific output

Answer: the third option. A head is a small add-on, usually one or a few layers, that turns the transformer's hidden states into the output a task needs. In my Assignment 5 GPT, `lm_head` is one linear layer from the 48-dimensional hidden state to the 50,257-token vocabulary. A sentiment classifier would swap in a linear layer to 2 classes. The Module 4 slides use the same word for RNN language models. There, "a predictive head" turns the latest hidden vector into a distribution over the vocabulary. The second option describes self-attention. The first isn't a real component.

13. What are the techniques to be aware of when batching sequences of different lengths together?
☒ Truncating ☐ Returning tensors ☒ Padding ☒ Attention masking

Answer: truncating, padding, and attention masking. Sequences in a batch have to stack into one rectangular tensor. Truncating cuts anything longer than the model's maximum length. Padding fills shorter sequences up to the longest one. Attention masking tells the model to ignore the pad positions so they don't change the real tokens' outputs. Returning tensors (`return_tensors="pt"`) only sets the output type, so it has nothing to do with uneven lengths.

The Assignment 5 code handles uneven lengths like this. `prepare_data` cuts each text into chunks of at most 32 tokens, the model's block size. `collate_fn` pads each batch with `PAD_ID` and builds a mask with 0 at the pads. `GPT.forward` multiplies the per-token loss by that mask. The mask never goes to the attention. The pads sit at the right end, and the causal mask already stops every real token from looking right.

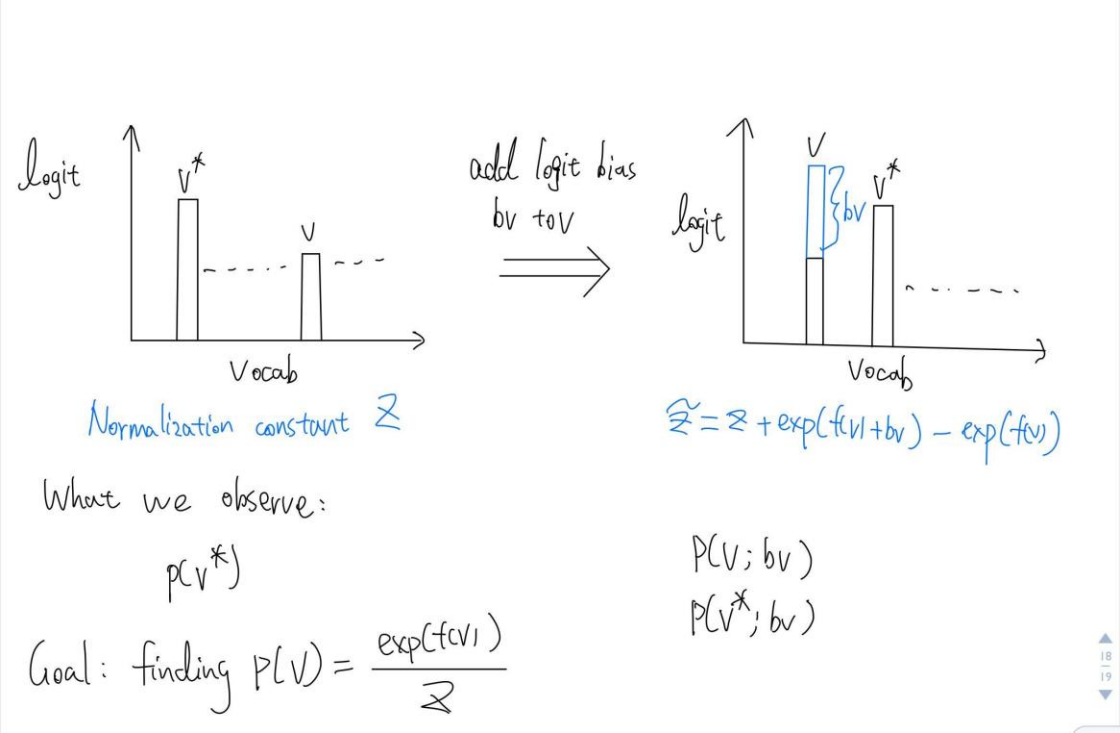
2 **Extra Credit:** Inferring the whole distribution from a semi-closed API

It's that time of the year! OpenAI frequently releases new language models that beat the hell out of all the language models out there. Jiefu, your boss at the startup is asking you to do cool stuff with this LM such as top-p sampling, but you realize that this API does not output the full probability distribution of the words, it only outputs the probabilities of the top-2 words. But you know that Jiefu does not take "no" for answer! So you need to figure out a way to identify the full distribution of probabilities over words.

All of a sudden, you realize that OpenAI has implemented a feature called "logit bias" that returns the distribution of the top-2 words after adding a bias value to a particular word in the **logit** distribution. This bias term is to the logits generated by the model prior to the normalization of the logits into a probability and returning the top-2 values. Logit bias is useful for up-weighting/down-weighting word probabilities in downstream applications that want a certain word to never/always appear in the output.

Your task is to find a way to logit bias identify the probability of all the words in the distribution. You algorithm needs accomplish this goal by at most $|V| + 1$ API calls (for a vocabulary of size $|V|$).

Hints: The figure below provides some hints into how to think about this problem. Initially (left) you observe v^* as the most likely word. You do not really observe the probability of v because the API does not expose it but you know that it's there in the distribution. Then since you want to identify the probability of v, you add b_v logit bias to this word to make it the highest probability word and you get a modified distribution (right). Notice that, these two distributions have different normalizing constants. If the left distribution has a normalizing constant of Z, the normalizing constant of the distribution on the right size would be $\sim Z = Z + exp(f(v) + b_v) - \exp\big(f(v)\big)$, where $f(v)$ is the original logit value of v. Also, notice that you observe the probability of v^* as $p(v^*)$ initially, and the modified probability of both v^* and v as $p(v^*; b_v)$ and $p(v; b_v)$ after logit bias added. Given these connections between the two distributions, you can infer $p(v) = \frac{f(v)}{Z}$.



Notation: $f(w)$ is the model's logit for word w, $Z = \sum_w \exp\big(f(w)\big)$, and $p(w) = \exp\big(f(w)\big)/Z$. (The hint's last line writes $p(v) = f(v)/Z$. The exponential is missing there.) The API returns the top 2 words with their probabilities, and it accepts a bias b added to one word's logit.

Algorithm:

1. Call the API once with no bias. It returns the most likely word v^* with $p(v^*)$ and the runner-up v_2 with $p(v_2)$.
2. For every other word v, call the API once with bias b on v only. Read $p(v^*; b)$ from the response, and $p(v; b)$ if v made the top 2.
3. Recover $p(v)$ with the formulas below.

That's $1 + (|V| - 2) = |V| - 1$ calls, under the $|V| + 1$ limit. The last word could even be skipped, since all the probabilities sum to 1.

Derivation. Adding b to v changes only v's term in the normalizer:

$$\tilde{Z} = Z - e^{f(v)} + e^{f(v)+b} = Z + e^{f(v)}(e^b - 1).$$

No other logit moved, so v^* is still above every word except possibly v. That means v^* is always in the top 2 of a biased call, and the API always shows $p(v^*; b) = e^{f(v^*)}/\tilde{Z}$. Dividing the two views of v^*:

$$\frac{p(v^*)}{p(v^*; b)} = \frac{e^{f(v^*)}/Z}{e^{f(v^*)}/\tilde{Z}} = \frac{\tilde{Z}}{Z} = 1 + \frac{e^{f(v)}}{Z}(e^b - 1) = 1 + p(v)(e^b - 1).$$

Solving for $p(v)$:

$$p(v) = \frac{p(v^*)/p(v^*; b) - 1}{e^b - 1}.$$

This works for any $b \neq 0$, and it only needs v^*'s probability.

If b is big enough that v also shows up in the top 2, the ratio of the two biased probabilities gives a second route:

$$\frac{p(v; b)}{p(v^*; b)} = \frac{e^{f(v)+b}}{e^{f(v^*)}} = e^b \frac{p(v)}{p(v^*)},$$

so

$$\log p(v) = \log p(v^*) - b + \log p(v; b) - \log p(v^*; b).$$

I'd use the second one in practice. The first subtracts two nearly equal numbers when $p(v)$ is tiny, so it loses digits.

Choosing b. For the second formula, v has to beat the runner-up after the bias, which means $e^b p(v) > p(v_2)$. I don't know $p(v)$ ahead of time, so I'd use the largest bias the API allows. OpenAI's `logit_bias` accepts values from -100 to 100 (OpenAI Help Center). With $b = 100$, any word with $p(v) > e^{-100} p(v_2)$ gets into the top 2. A word that still doesn't show up has $p(v) < e^{-100} p(v_2) \approx 3.7 \times 10^{-44}\, p(v_2)$, and the first formula still gives its value, up to floating-point precision. In double precision, $p(v)(e^{100} - 1)$ gets lost next to the 1 once $p(v)$ drops below about 10^{-59}, so those words come out as 0.

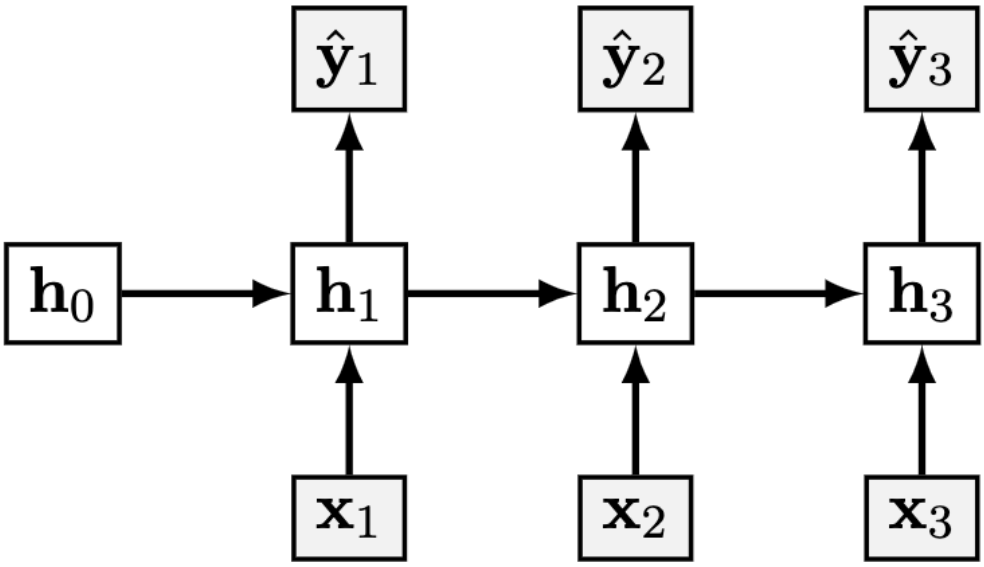
Check. I tested this against a simulated API that holds 1,000 random logits and only ever returns the top 2 (`verification/verify_theory.py`). The ratio formula with $b = 40$ recovered every probability to within 10^{-16} in 999 calls, and the recovered distribution sums to 1. The first formula with $b = 2$ was also within 1.04×10^{-16} in absolute terms, but its relative error on the rarest words was 1.55×10^{-8}. That's the lost digits I mentioned.

With all $|V|$ probabilities in hand, Jiefu gets his top-p sampling. Sort them, keep the smallest set whose total reaches p, renormalize, and sample. Carlini et al. (2024) used the same logit-bias idea against production APIs to rebuild full logit vectors from top-K log-

probabilities. One assumption under all of this is that the API returns the same logits for the same prompt on every call. If it added noise or changed models between calls, the recovered numbers would be off by that much.

3 Training a Recurrent Neural Net

Consider the following simple RNN architecture:



Where the layers and their corresponding weights are given below:

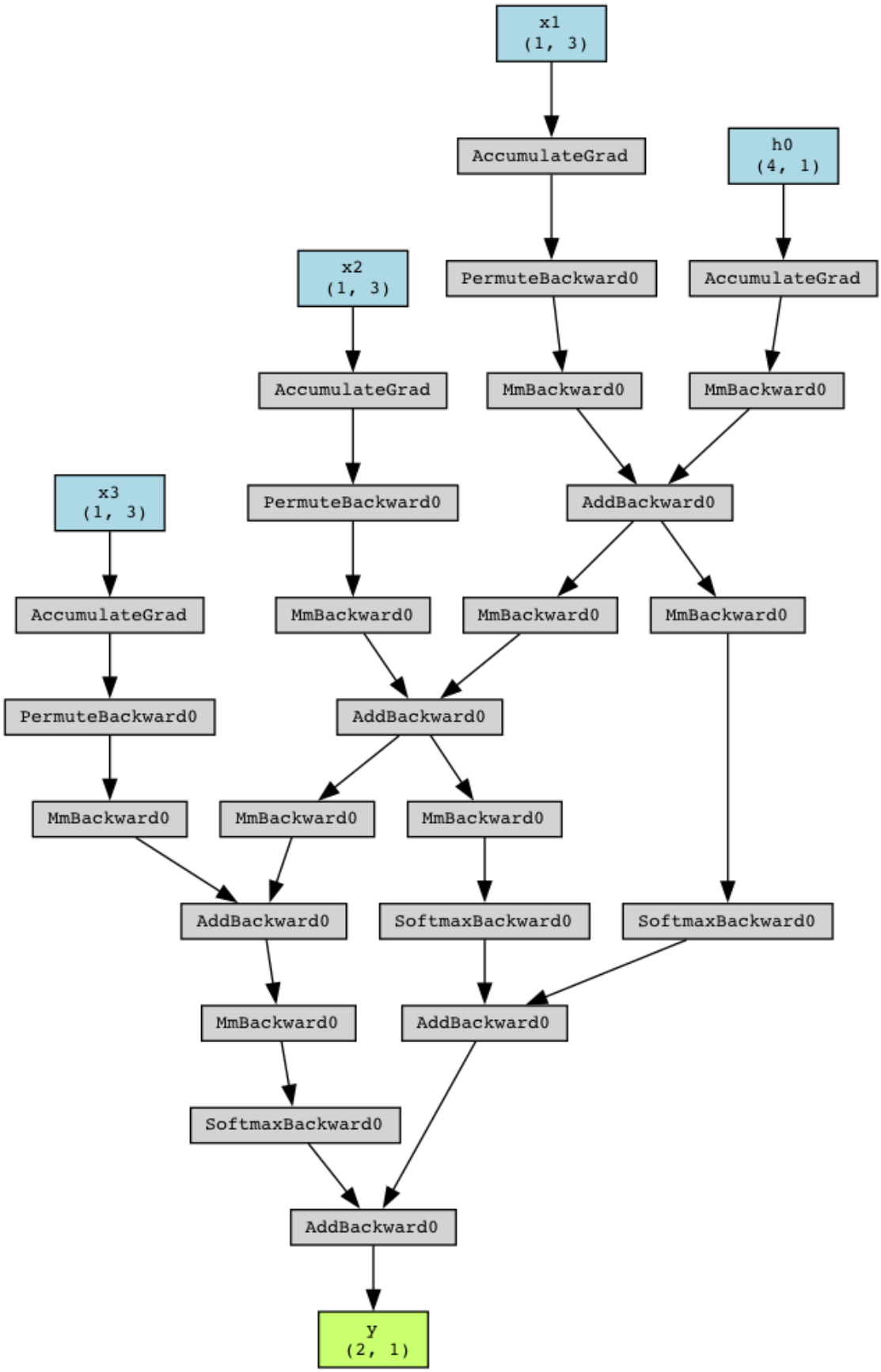
$$\begin{array}{llll} \mathbf{x}_t & \in \mathbb{R}^3 & \mathbf{W}_{hx} & \in \mathbb{R}^{4\times 3} \\ \mathbf{h}_t & \in \mathbb{R}^4 & \mathbf{W}_{yh} & \in \mathbb{R}^{2\times 4} \\ \mathbf{y}_t, \hat{\mathbf{y}}_t & \in \mathbb{R}^2 & \mathbf{W}_{hh} & \in \mathbb{R}^{4\times 4} \end{array}$$

$$\begin{aligned} J &= -\sum_{t=1}^{3}\sum_{i=1}^{2} y_{t,i}\log\left(\hat{y}_{t,i}\right) \\ \hat{\mathbf{y}}_t &= \sigma(\mathbf{o}_t) \\ \mathbf{o}_t &= \mathbf{W}_{yh}\mathbf{h}_t \\ \mathbf{h}_t &= \psi(\mathbf{z}_t) \\ \mathbf{z}_t &= \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t \end{aligned}$$

Where σ is the softmax activation and ψ is the identity activation (i.e. no activation).

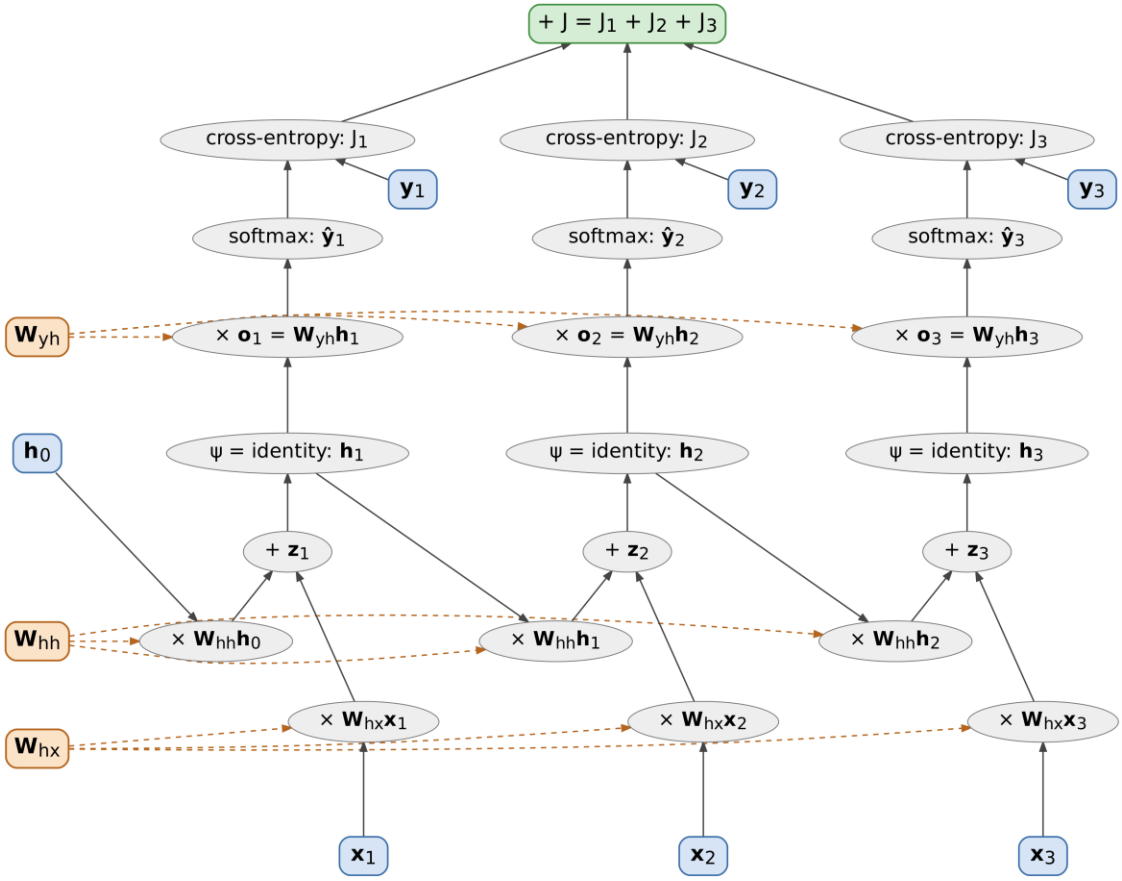
3.1 Computation Graph

Draw the computational graph for the given model.


x1
(1, 3)
AccumulateGrad
h0
(4, 1)
x2
(1, 3)
PermuteBackward0
AccumulateGrad
AccumulateGrad
MmBackward0
MmBackward0
x3
(1, 3)
PermuteBackward0
AddBackward0
AccumulateGrad
MmBackward0
MmBackward0
MmBackward0
PermuteBackward0
AddBackward0
MmBackward0
MmBackward0
MmBackward0
AddBackward0
SoftmaxBackward0
SoftmaxBackward0
MmBackward0
AddBackward0
SoftmaxBackward0
AddBackward0
y
(2, 1)

Q2.1 Computation graph.

Answer: my graph is the figure below. The torchviz figure above it, captioned "Q2.1 Computation graph," came with the assignment.



Computation graph of the unrolled RNN for $T = 3$. Blue boxes are inputs, targets, and the initial state. Orange boxes are the three weight matrices. Each one is a single node that feeds all three time steps (dashed edges). Gray ovals are operations, and the green box is the total loss.

Each time step runs the same chain. Two matrix-vector products, $\mathbf{W}_{hh}\mathbf{h}_{t-1}$ and $\mathbf{W}_{hx}\mathbf{x}_t$, add up to $\mathbf{z}_t$. The identity ψ gives $\mathbf{h}_t$. Then $\mathbf{o}_t = \mathbf{W}_{yh}\mathbf{h}_t$, the softmax gives $\hat{\mathbf{y}}_t$, and the cross-entropy against $\mathbf{y}_t$ gives J_t. The three J_t add up to J.

Information crosses time in two ways. $\mathbf{h}_t$ feeds the next step's product $\mathbf{W}_{hh}\mathbf{h}_t$, and each weight matrix feeds every step. Backprop follows both. That's why 3.2.2 has powers of $\mathbf{W}_{hh}$ and 3.2.3 has a sum over steps. The graph source is `writeup/figures/rnn_graph.dot` in the repository.

3.2 Backprop for RNN

Now you will derive the steps of the backpropagation algorithm that lead to the computation of $\frac{dJ}{dW_{hh}}$. For all parts of this question, please write your answer in terms of W_{hh}, W_{yh}, y, $\hat{y}$, h, and any additional terms specified in the question (note: this does not mean that every term listed

shows up in every answer, but rather that you should simplify terms into these as much as possible when you can).

> Conventions for all four parts: the derivative of the scalar J_t with respect to a vector or matrix has the same shape as that vector or matrix. Vectors are columns. $\mathbf{y}_t$ is the one-hot target. The derivation only uses $\sum_i y_{t,i} = 1$, so a soft target works too. ψ is the identity, so $\mathbf{h}_t = \mathbf{z}_t$. $\mathbf{h}_0$ is given and fixed. Every result below matches PyTorch autograd on random weights with a nonzero $\mathbf{h}_0$ (`verification/verify_theory.py`).

1. Let's define a cross-entropy for our RNN at time t: $J_t = -\sum_{i=1}^{2} y_{t,i} \log(\hat{y}_{t,i})$. What is $\frac{\partial J_t}{\partial o_t}$? Write your answer and show your work. **Hint:** First derive the partial derivatives with respective to everything after h_t. After that, try to break up the objective into a term for each timestep.

> Answer:
>
> $$\frac{\partial J_t}{\partial \mathbf{o}_t} = \hat{\mathbf{y}}_t - \mathbf{y}_t.$$
>
> Work, from the loss back to $\mathbf{h}_t$.
>
> Step 1, the loss with respect to $\hat{\mathbf{y}}_t$. Since $J_t = -\sum_i y_{t,i} \log \hat{y}_{t,i}$,
>
> $$\frac{\partial J_t}{\partial \hat{y}_{t,i}} = -\frac{y_{t,i}}{\hat{y}_{t,i}}.$$
>
> Step 2, the softmax with respect to $\mathbf{o}_t$. With $\hat{y}_{t,i} = e^{o_{t,i}} / \sum_k e^{o_{t,k}}$, the quotient rule gives
>
> $$\frac{\partial \hat{y}_{t,i}}{\partial o_{t,j}} = \frac{\delta_{ij}\, e^{o_{t,i}} \sum_k e^{o_{t,k}} - e^{o_{t,i}}\, e^{o_{t,j}}}{(\sum_k e^{o_{t,k}})^2} = \hat{y}_{t,i}(\delta_{ij} - \hat{y}_{t,j}).$$
>
> As a matrix, $\frac{\partial \hat{\mathbf{y}}_t}{\partial \mathbf{o}_t} = \mathrm{diag}(\hat{\mathbf{y}}_t) - \hat{\mathbf{y}}_t \hat{\mathbf{y}}_t^\top$, which is 2×2.
>
> Step 3, the chain rule:
>
> $$\frac{\partial J_t}{\partial o_{t,j}} = \sum_i \frac{\partial J_t}{\partial \hat{y}_{t,i}} \frac{\partial \hat{y}_{t,i}}{\partial o_{t,j}} = \sum_i \left(-\frac{y_{t,i}}{\hat{y}_{t,i}}\right) \hat{y}_{t,i}(\delta_{ij} - \hat{y}_{t,j}) = -y_{t,j} + \hat{y}_{t,j} \sum_i y_{t,i} = \hat{y}_{t,j} - y_{t,j}.$$
>
> So $\mathbf{g}_{\mathbf{o}_t} = \hat{\mathbf{y}}_t - \mathbf{y}_t$, a 2×1 vector.
>
> Step 4, the rest of what sits after $\mathbf{h}_t$. Since $\mathbf{o}_t = \mathbf{W}_{yh}\mathbf{h}_t$, the Jacobian $\partial \mathbf{o}_t / \partial \mathbf{h}_t$ is $\mathbf{W}_{yh}$, so
>
> $$\frac{\partial J_t}{\partial \mathbf{h}_t} = \mathbf{W}_{yh}^\top(\hat{\mathbf{y}}_t - \mathbf{y}_t)\ (4 \times 1), \qquad \frac{\partial J_t}{\partial \mathbf{W}_{yh}} = (\hat{\mathbf{y}}_t - \mathbf{y}_t)\mathbf{h}_t^\top\ (2 \times 4).$$
>
> Step 5, one term per time step. $J = J_1 + J_2 + J_3$. $\mathbf{o}_t$ only feeds $\hat{\mathbf{y}}_t$, and $\hat{\mathbf{y}}_t$ only appears in J_t, so $\partial J / \partial \mathbf{o}_t = \partial J_t / \partial \mathbf{o}_t = \hat{\mathbf{y}}_t - \mathbf{y}_t$. $\mathbf{W}_{yh}$ is used at all three steps, so $\partial J / \partial \mathbf{W}_{yh} = \sum_t (\hat{\mathbf{y}}_t - \mathbf{y}_t)\, \mathbf{h}_t^\top$. $\mathbf{h}_t$ is different. It feeds $\mathbf{o}_t$ and also $\mathbf{z}_{t+1}$, so later losses depend on it too. That's the path 3.2.2 follows.

2. Suppose you have a variable $\mathbf{g}_{\mathbf{o}_t}$ that stores the value of $\frac{\partial J_t}{\partial \mathbf{o}_t}$. What is $\frac{\partial J_t}{\partial \mathbf{h}_i}$ for an arbitrary $i \in [1,3]$ ($i \leq t$)? Write your solution in terms of the $\mathbf{g}_{\mathbf{o}_t}$ and the aforementioned variables and show your work.

Answer: for $i \leq t$,

$$\frac{\partial J_t}{\partial \mathbf{h}_i} = (\mathbf{W}_{hh}^\top)^{t-i} \mathbf{W}_{yh}^\top \mathbf{g}_{\mathbf{o}_t} = (\mathbf{W}_{hh}^\top)^{t-i} \mathbf{W}_{yh}^\top (\hat{\mathbf{y}}_t - \mathbf{y}_t).$$

Work. J_t depends on $\mathbf{h}_i$ only through the chain $\mathbf{h}_i \to \mathbf{z}_{i+1} \to \mathbf{h}_{i+1} \to \cdots \to \mathbf{h}_t \to \mathbf{o}_t \to J_t$. $\mathbf{h}_i$ also feeds $\mathbf{o}_i$, but $\mathbf{o}_i$ only reaches J_i, so for $i < t$ that path adds nothing to J_t.

One step of the chain: $\mathbf{h}_k = \psi(\mathbf{z}_k) = \mathbf{W}_{hh}\mathbf{h}_{k-1} + \mathbf{W}_{hx}\mathbf{x}_k$, so the Jacobian is

$$\frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}} = \operatorname{diag}\big(\psi'(\mathbf{z}_k)\big)\mathbf{W}_{hh} = \mathbf{W}_{hh}, \quad \text{since } \psi' = 1.$$

Start at $i = t$, using 3.2.1:

$$\frac{\partial J_t}{\partial \mathbf{h}_t} = \left(\frac{\partial \mathbf{o}_t}{\partial \mathbf{h}_t}\right)^\top \mathbf{g}_{\mathbf{o}_t} = \mathbf{W}_{yh}^\top \mathbf{g}_{\mathbf{o}_t}.$$

Step back one position at a time:

$$\frac{\partial J_t}{\partial \mathbf{h}_{k-1}} = \left(\frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}}\right)^\top \frac{\partial J_t}{\partial \mathbf{h}_k} = \mathbf{W}_{hh}^\top \frac{\partial J_t}{\partial \mathbf{h}_k}.$$

After $t - i$ steps back from $\mathbf{h}_t$ this gives the answer. Shapes check out: $\mathbf{W}_{yh}^\top \mathbf{g}_{\mathbf{o}_t}$ is $(4 \times 2)(2 \times 1) = 4 \times 1$, and each $\mathbf{W}_{hh}^\top$ keeps it 4×1.

All six cases for $T = 3$:

$$t = 1: \quad \frac{\partial J_1}{\partial \mathbf{h}_1} = \mathbf{W}_{yh}^\top \mathbf{g}_{\mathbf{o}_1}$$

$$t = 2: \quad \frac{\partial J_2}{\partial \mathbf{h}_2} = \mathbf{W}_{yh}^\top \mathbf{g}_{\mathbf{o}_2}, \qquad \frac{\partial J_2}{\partial \mathbf{h}_1} = \mathbf{W}_{hh}^\top \mathbf{W}_{yh}^\top \mathbf{g}_{\mathbf{o}_2}$$

$$t = 3: \quad \frac{\partial J_3}{\partial \mathbf{h}_3} = \mathbf{W}_{yh}^\top \mathbf{g}_{\mathbf{o}_3}, \qquad \frac{\partial J_3}{\partial \mathbf{h}_2} = \mathbf{W}_{hh}^\top \mathbf{W}_{yh}^\top \mathbf{g}_{\mathbf{o}_3}, \qquad \frac{\partial J_3}{\partial \mathbf{h}_1} = (\mathbf{W}_{hh}^\top)^2 \mathbf{W}_{yh}^\top \mathbf{g}_{\mathbf{o}_3}$$

The power $(\mathbf{W}_{hh}^\top)^{t-i}$ is where exploding and vanishing gradients come from (question 7 in Section 1).

3. Suppose you have a variable $\mathbf{g}_{\mathbf{h}_i}$ that stores the value of $\frac{\partial J_t}{\partial \mathbf{h}_i}$. What is $\frac{\partial J_t}{\partial \mathbf{W}_{hh}}$? Write your solution in terms of the $\mathbf{g}_{\mathbf{h}_i}$ and the aforementioned variables. Show your work in the second.

Answer:

$$\frac{\partial J_t}{\partial \mathbf{W}_{hh}} = \sum_{i=1}^{t} \mathbf{g}_{\mathbf{h}_i} \, \mathbf{h}_{i-1}^\top = \sum_{i=1}^{t} (\mathbf{W}_{hh}^\top)^{t-i} \, \mathbf{W}_{yh}^\top (\hat{\mathbf{y}}_t - \mathbf{y}_t) \mathbf{h}_{i-1}^\top.$$

Work. $\mathbf{W}_{hh}$ is used at every step $i = 1, \ldots, t$ in $\mathbf{z}_i = \mathbf{W}_{hh}\mathbf{h}_{i-1} + \mathbf{W}_{hx}\mathbf{x}_i$. Steps after t don't reach J_t. I treat each use as its own copy $\mathbf{W}^{(i)}$, all equal to $\mathbf{W}_{hh}$. By the multivariable chain rule, the derivative with respect to the shared matrix is the sum over the copies:

$$\frac{\partial J_t}{\partial \mathbf{W}_{hh}} = \sum_{i=1}^{t} \frac{\partial J_t}{\partial \mathbf{W}^{(i)}}.$$

This is the "trace the derivative all the way back to the beginning, while summing the results" step on the Module 4 backprop-through-time slides (74 and 75).

One copy. Entry by entry, $z_{i,c} = \sum_b W_{cb}^{(i)}\, h_{i-1,b} + (\mathbf{W}_{hx}\mathbf{x}_i)_c$, so $\partial z_{i,c} / \partial W_{ab}^{(i)} = \delta_{ca}\, h_{i-1,b}$. Then

$$\frac{\partial J_t}{\partial W_{ab}^{(i)}} = \sum_c \frac{\partial J_t}{\partial z_{i,c}}\, \delta_{ca}\, h_{i-1,b} = \left(\frac{\partial J_t}{\partial \mathbf{z}_i}\right)_a h_{i-1,b}, \qquad \text{so} \qquad \frac{\partial J_t}{\partial \mathbf{W}^{(i)}} = \frac{\partial J_t}{\partial \mathbf{z}_i}\, \mathbf{h}_{i-1}^{\top}.$$

Since ψ is the identity, $\mathbf{h}_i = \mathbf{z}_i$ and $\partial J_t / \partial \mathbf{z}_i = \partial J_t / \partial \mathbf{h}_i = \mathbf{g}_{\mathbf{h}_i}$. (With another ψ it would be $\psi'(\mathbf{z}_i) \odot \mathbf{g}_{\mathbf{h}_i}$.) Each copy gives the outer product $\mathbf{g}_{\mathbf{h}_i}\mathbf{h}_{i-1}^{\top}$, a 4×1 times a 1×4, and the sum over copies is the answer. Plugging in 3.2.2 gives the second form.

For $t = 3$, written out:

$$\frac{\partial J_3}{\partial \mathbf{W}_{hh}} = \mathbf{g}_{\mathbf{h}_3}\mathbf{h}_2^{\top} + \mathbf{g}_{\mathbf{h}_2}\mathbf{h}_1^{\top} + \mathbf{g}_{\mathbf{h}_1}\mathbf{h}_0^{\top}.$$

If $\mathbf{h}_0 = \mathbf{0}$, the last term drops out.

4. Suppose you have a variable $\mathbf{g}_{W_{hh},t}$ that stores the value of $\frac{\partial J_t}{\partial W_{hh}}$. What is $\frac{\partial J}{\partial \mathbf{W}_{hh}}$? Write your solution in terms of the $\mathbf{g}_{W_{hh},t}$ and the aforementioned variables. Show your work.

Answer:

$$\frac{\partial J}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^{3} \mathbf{g}_{W_{hh},t}.$$

Work. $J = -\sum_{t=1}^{3}\sum_{i=1}^{2} y_{t,i} \log \hat{y}_{t,i} = J_1 + J_2 + J_3$, and the derivative of a sum is the sum of the derivatives, so $\partial J / \partial \mathbf{W}_{hh} = \sum_t \partial J_t / \partial \mathbf{W}_{hh} = \sum_t \mathbf{g}_{W_{hh},t}$.

Written out with 3.2.2 and 3.2.3, that's six terms:

$$\begin{aligned}
\frac{\partial J}{\partial \mathbf{W}_{hh}} &= \sum_{t=1}^{3}\sum_{i=1}^{t} (\mathbf{W}_{hh}^{\top})^{t-i}\, \mathbf{W}_{yh}^{\top}(\hat{\mathbf{y}}_t - \mathbf{y}_t)\mathbf{h}_{i-1}^{\top} \\
&= \mathbf{W}_{yh}^{\top}(\hat{\mathbf{y}}_1 - \mathbf{y}_1)\mathbf{h}_0^{\top} \\
&\quad + \mathbf{W}_{yh}^{\top}(\hat{\mathbf{y}}_2 - \mathbf{y}_2)\mathbf{h}_1^{\top} + \mathbf{W}_{hh}^{\top}\mathbf{W}_{yh}^{\top}(\hat{\mathbf{y}}_2 - \mathbf{y}_2)\mathbf{h}_0^{\top} \\
&\quad + \mathbf{W}_{yh}^{\top}(\hat{\mathbf{y}}_3 - \mathbf{y}_3)\mathbf{h}_2^{\top} + \mathbf{W}_{hh}^{\top}\mathbf{W}_{yh}^{\top}(\hat{\mathbf{y}}_3 - \mathbf{y}_3)\mathbf{h}_1^{\top} + (\mathbf{W}_{hh}^{\top})^2\mathbf{W}_{yh}^{\top}(\hat{\mathbf{y}}_3 - \mathbf{y}_3)\mathbf{h}_0^{\top}
\end{aligned}$$

Backprop doesn't build the six terms one at a time. Grouping them by i instead of t gives one backward sweep. Let $\boldsymbol{\delta}_i = \partial J / \partial \mathbf{h}_i$, which collects every J_t with $t \geq i$:

$$
\begin{aligned}
\boldsymbol{\delta}_3 &= \mathbf{W}_{yh}^{\top}(\hat{\mathbf{y}}_3 - \mathbf{y}_3) \\
\boldsymbol{\delta}_i &= \mathbf{W}_{yh}^{\top}(\hat{\mathbf{y}}_i - \mathbf{y}_i) + \mathbf{W}_{hh}^{\top}\boldsymbol{\delta}_{i+1} \quad \text{for } i = 2,1 \\
\frac{\partial J}{\partial \mathbf{W}_{hh}} &= \sum_{i=1}^{3} \boldsymbol{\delta}_i \, \mathbf{h}_{i-1}^{\top}
\end{aligned}
$$

Expanding $\boldsymbol{\delta}_1$, $\boldsymbol{\delta}_2$, and $\boldsymbol{\delta}_3$ gives back the same six terms. The sweep takes $O(T)$ matrix-vector products instead of $O(T^2)$. Both forms match autograd, with largest differences of 8.9×10^{-16} and 2.2×10^{-16}.

4 Self-Attention and Transformers

Recall that the transformer architecture uses scaled dot-product attention to compute *attention weights*:

$$\boldsymbol{\alpha}^{(t)} = \text{softmax}\left(\frac{\mathsf{q}_t \mathsf{K}^{\top}}{\sqrt{h}}\right) \in [0,1]^n$$

The resulting embedding in the output of attention at position t are:

$$\text{Attention}(\mathsf{Q},\mathsf{K},\mathsf{V})_t = \sum_{t\prime=1}^{n} \alpha_{t\prime}^{(t)} \mathsf{v}_{t\prime} \in \mathbb{R}^{1\times h},$$

where $\boldsymbol{\alpha}^{(t)} = \left[\alpha_0^{(t)}, \ldots, \alpha_n^{(t)}\right]$. The same idea can be stated in a matrix form,

$$\text{Attention}(\mathsf{Q},\mathsf{K},\mathsf{V}) = \text{softmax}\left(\frac{\mathsf{Q}\mathsf{K}^{\top}}{\sqrt{h}}\right)\mathsf{V} \in \mathbb{R}^{n\times h}.$$

In the above equations:

- h is the hidden state dimension and n is the input sequence length;
- $\mathsf{X} \in \mathbb{R}^{n\times h}$ is the input to the attention;
- $\mathsf{x}_t \in \mathbb{R}^{1\times h}$ is the slice of X at position t, i.e. vector representation (embedding) of the input token at position t;
- $\mathsf{W}_q, \mathsf{W}_k, \mathsf{W}_v \in \mathbb{R}^{h\times h}$ are the projection matrices to build query, key and value representations;
- $\mathsf{Q} = \mathsf{X}\mathsf{W}_q \in \mathbb{R}^{n\times h}$,$\mathsf{K} = \mathsf{X}\mathsf{W}_k \in \mathbb{R}^{n\times h}$,$\mathsf{V} = \mathsf{X}\mathsf{W}_v \in \mathbb{R}^{n\times h}$ are the query, key and value representations;
- $\mathsf{q}_t = \mathsf{x}_t\mathsf{W}_q \in \mathbb{R}^{1\times h}$ is the slice of Q at position t. Similarly, $\mathsf{k}_t = \mathsf{x}_t\mathsf{W}_k \in \mathbb{R}^{1\times h}$ and $\mathsf{v}_t = \mathsf{x}_t\mathsf{W}_v \in \mathbb{R}^{1\times h}$.

Now answer the following questions:

4.1 Complexity

What is the computational complexity of self-attention layer in terms of n and h? In particular, show that the complexity of a self-attention layer at test-time scales quadratically with the sequence length n. Lastly, explain (no more than 5 sentences) why this can be computed efficiently on GPUs, despite being a quadratic function of sequence length n.

Answer: one self-attention layer takes $O(nh^2 + n^2h)$ time and $O(n^2 + nh)$ memory. A trained model's width h is fixed at test time, so the n^2h term makes the cost quadratic in n.

Work, counting multiply-adds:

1. Projections $\mathbf{Q} = \mathbf{X}\mathbf{W}_q$, $\mathbf{K} = \mathbf{X}\mathbf{W}_k$, $\mathbf{V} = \mathbf{X}\mathbf{W}_v$. Each is $(n \times h)(h \times h)$, so nh^2. All three take $3nh^2$.
2. Scores $\mathbf{Q}\mathbf{K}^\top/\sqrt{h}$. $(n \times h)(h \times n)$ is n^2 dot products of length h, so n^2h. The scaling adds n^2.
3. The softmax along each row of the $n \times n$ scores. That's an exp, a sum, and a divide per entry, so $O(n^2)$.
4. The output, softmax times $\mathbf{V}$. $(n \times n)(n \times h)$ is another n^2h.

$$\text{Total} = 3nh^2 + 2n^2h + O(n^2) = O(nh^2 + n^2h).$$

The $n \times n$ weight matrix also has to sit in memory, which is the $O(n^2)$ memory term.

Why it's quadratic in n at test time. With h fixed, the cost is $an^2 + bn$ with $a = 2h + c$, where c covers the scaling and softmax. Doubling n about quadruples the leading term. That term is bigger than the projections once $2n^2h > 3nh^2$, which means $n > 1.5h$. For $h = 64$ at $n = 128$, the projections take 1.57 million multiply-adds and the scores plus output take 2.10 million. At $n = 1{,}024$ it's 12.6 million against 134 million (`verification/verify_theory.py`).

Generating one token at a time doesn't get around it. With the earlier keys and values cached, token t needs $\mathbf{q}_t\mathbf{K}^\top$ and $\boldsymbol{\alpha}^{(t)}\mathbf{V}$ over t positions, which is $O(th)$. Summed over $t = 1, \ldots, n$, that's $O(n^2h)$.

I also timed one layer at $h = 64$ on my laptop (`verification/attention_scaling.py`):

n	Multiply-adds, $3nh^2 + 2n^2h$	CPU (ms)	GPU (ms)	CPU / GPU
512	39,845,888	1.18	0.131	9.0
1,024	146,800,640	2.81	0.257	10.9
2,048	562,036,736	6.95	0.633	11.0
4,096	2,197,815,296	26.11	3.240	8.1
8,192	8,690,597,888	94.53	12.587	7.5

From $n = 4{,}096$ to $8{,}192$ the CPU time grew 3.6 times and the GPU time 3.9 times, close to the 4 times a quadratic predicts.

Why it's still fast on a GPU (5 sentences): Each of the n^2 scores $\mathbf{q}_t \cdot \mathbf{k}_{t'}$ can be computed without waiting on any other, unlike an RNN, where $\mathbf{h}_t$ waits for $\mathbf{h}_{t-1}$. So the layer is a few big matrix multiplies, and a GPU runs those across thousands of cores at once, for every head and sequence. The work is still quadratic, but the time is that work divided by a very large throughput, as the Module 5 slides note. On my laptop, one layer ran 7.5 to 11 times faster on the RTX 4050 than on the 16-core CPU. Training the Assignment 5 GPT for 50 steps ran 7.8 to 14.7 times faster on the GPU across five runs (Section 5.1.9).

4.2 Masking in Self-Attention

Suppose we are are using token-making objective to create a language generation model that uses self-attention. For example, suppose we want to mask $t = 3$ position. Then, it does not make sense for q_t: $\forall t \in [0 \ldots n]$ to look at k_3 and v_3. Describe one way we can go about implementing such masking.

Answer: add a mask to the attention scores before the softmax, with $-\infty$ in column 3 and 0 everywhere else. For $n = 5$ positions (0 to 4):

$$\mathbf{M} = \begin{bmatrix} 0 & 0 & 0 & -\infty & 0 \\ 0 & 0 & 0 & -\infty & 0 \\ 0 & 0 & 0 & -\infty & 0 \\ 0 & 0 & 0 & -\infty & 0 \\ 0 & 0 & 0 & -\infty & 0 \end{bmatrix}, \qquad \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{h}} + \mathbf{M}\right)\mathbf{V}.$$

Why it works. Write $s_{tj} = \mathbf{q}_t \cdot \mathbf{k}_j / \sqrt{h}$. For any query t, the weight on position 3 becomes

$$\alpha_3^{(t)} = \frac{\exp(s_{t3} - \infty)}{\sum_j \exp\left(s_{tj} + M_{tj}\right)} = 0,$$

and the other weights renormalize over the keys that are left:

$$\alpha_{t'}^{(t)} = \frac{\exp(s_{tt'})}{\sum_{j \neq 3} \exp\left(s_{tj}\right)}, \qquad \text{Attention}_t = \sum_{t' \neq 3} \alpha_{t'}^{(t)}\, \mathbf{v}_{t'}.$$

So no query's weights use $\mathbf{k}_3$, no output sums in $\mathbf{v}_3$, and no gradient flows back into either.

In code it's the same call as the causal mask in my Assignment 5 `model.py`, with a different pattern:

```
keep = torch.ones(n, n, dtype=torch.bool)
keep[:, 3] = False                          # no query may look at key 3
att = att.masked_fill(~keep, float('-inf'))  # att holds Q K^T / sqrt(h)
att = F.softmax(att, dim=-1)
```

To combine it with the causal mask, AND the two boolean masks. Putting $-\infty$ in before the softmax beats zeroing weights after it, because the softmax then renormalizes the

rest for free. It also keeps every tensor the same shape, which matters in a batch where different sequences mask different positions.

Two things to watch. The query at position 3 still comes from $\mathbf{x}_3$. For a masked-token objective I'd also swap $\mathbf{x}_3$ for a `[MASK]` embedding, as BERT does. Otherwise the residual connection carries token 3 straight to the output at position 3. Also, if a mask ever removes every key in a row, the softmax of all $-\infty$ is NaN. Each row needs at least one visible key. Under a causal mask, position 0 can only see itself, so masking position 0 would cause that.

Check (`verification/verify_theory.py`): column 3 gets weight 0 and every row still sums to 1. Changing $\mathbf{x}_3$ leaves the outputs at every other position unchanged. Row 3 changes because its own query changed.

4.3 **Extra Credit:** Linear Attention with SVD

It has been empirically shown in Transformer models that the context mapping matrix $P = \text{softmax}\left(\frac{QK^\top}{\sqrt{h}}\right)$ often has a low rank. Show that if the rank of P is k and we already have access to the SVD of P, then it is possible to compute self-attention in $O(nkh)$ time.

Answer: write the SVD in thin form and multiply from the right, so the $n \times n$ matrix is never formed.

$\mathbf{P}$ has rank k, so it has k nonzero singular values and its thin SVD is

$$\mathbf{P} = \mathbf{U}\mathbf{\Sigma}\mathbf{R}^\top, \qquad \mathbf{U} \in \mathbb{R}^{n\times k},\ \mathbf{\Sigma} = \text{diag}(\sigma_1, \dots, \sigma_k) \in \mathbb{R}^{k\times k},\ \mathbf{R} \in \mathbb{R}^{n\times k}.$$

(I use $\mathbf{R}$ for the right singular vectors so it doesn't clash with the values $\mathbf{V}$.) The attention output is $\mathbf{PV}$ with $\mathbf{V} \in \mathbb{R}^{n\times h}$. Matrix products are associative, so

$$\mathbf{PV} = \mathbf{U}\big(\mathbf{\Sigma}(\mathbf{R}^\top\mathbf{V})\big).$$

Cost, step by step:

1. $\mathbf{R}^\top\mathbf{V}$ is $(k \times n)(n \times h)$, so nkh multiply-adds. The result is $k \times h$.
2. $\mathbf{\Sigma}(\mathbf{R}^\top\mathbf{V})$ scales k rows of length h, so kh.
3. $\mathbf{U}(\cdot)$ is $(n \times k)(k \times h)$, so nkh. The result is $n \times h$.

$$\text{Total} = 2nkh + kh = O(nkh).$$

Multiplying $\mathbf{PV}$ directly costs n^2h, so this saves a factor of about $n/(2k)$. The order matters. $(\mathbf{U}\mathbf{\Sigma}\mathbf{R}^\top)\mathbf{V}$ would build the $n \times n$ matrix first and cost $O(n^2k + n^2h)$.

This takes the SVD as given, as the question says. Building $\mathbf{P}$ and its SVD would cost more than the attention itself. That's why 4.4 and Linformer (Wang et al., 2020) project the keys and values instead.

Check: with $n = 300$, $h = 64$, $k = 12$, $\mathbf{U}\big(\mathbf{\Sigma}(\mathbf{R}^\top\mathbf{V})\big)$ matches $\mathbf{PV}$ to within 4.0×10^{-15} (`verification/verify_theory.py`).

4.4 **Extra Credit:** Linear Attention by Projection

Suppose we ignore the Softmax and scaling and let $P = \frac{QK^\top}{\sqrt{h}} \in \mathbb{R}^{n\times n}$. Assume P is rank k. Show that there exist two linear projection matrices C,D $\in \mathbb{R}^{k\times n}$ such that $PV = Q(CK)^\top DV$ and the right hand side can be computed in $O(nkh)$ time. **Hint:** Consider using SVD in your proof.

Answer: take the thin SVD $\mathbf{P} = \mathbf{U\Sigma R}^\top$ and set $\mathbf{C} = \mathbf{D} = \mathbf{R}^\top \in \mathbb{R}^{k\times n}$. Then $\mathbf{PV} = \mathbf{Q(CK)}^\top\mathbf{DV}$, and computing it in the order $(\mathbf{Q(CK)}^\top)(\mathbf{DV})$ costs $O(nkh)$.

Proof. $\mathbf{P}$ has rank k, so its thin SVD is $\mathbf{P} = \mathbf{U\Sigma R}^\top$ with $\mathbf{U}, \mathbf{R} \in \mathbb{R}^{n\times k}$ and $\mathbf{R}^\top\mathbf{R} = \mathbf{I}_k$, since the columns of $\mathbf{R}$ are orthonormal.

Step 1, $\mathbf{PRR}^\top = \mathbf{P}$:

$$\mathbf{PRR}^\top = \mathbf{U\Sigma}(\mathbf{R}^\top\mathbf{R})\mathbf{R}^\top = \mathbf{U\Sigma}\,\mathbf{I}_k\,\mathbf{R}^\top = \mathbf{P}.$$

In words, $\mathbf{RR}^\top$ projects onto the row space of $\mathbf{P}$, and the rows of $\mathbf{P}$ are already in it.

Step 2, plug in $\mathbf{C} = \mathbf{D} = \mathbf{R}^\top$, using $(\mathbf{R}^\top\mathbf{K})^\top = \mathbf{K}^\top\mathbf{R}$ and $\mathbf{P} = \mathbf{QK}^\top$:

$$\mathbf{Q(CK)}^\top\mathbf{DV} = \mathbf{Q}(\mathbf{R}^\top\mathbf{K})^\top\mathbf{R}^\top\mathbf{V} = \mathbf{QK}^\top\mathbf{RR}^\top\mathbf{V} = \mathbf{PRR}^\top\mathbf{V} = \mathbf{PV}.$$

The question says to ignore the scaling, but its formula for $\mathbf{P}$ still has $1/\sqrt{h}$. If it stays, fold the scalar into $\mathbf{C}$ by setting $\mathbf{C} = \mathbf{R}^\top/\sqrt{h}$ and $\mathbf{D} = \mathbf{R}^\top$. Here $\mathbf{R}$ comes from the SVD of the scaled $\mathbf{P}$, which has the same singular vectors. Then $\mathbf{Q(CK)}^\top\mathbf{DV} = \frac{1}{\sqrt{h}}\mathbf{QK}^\top\mathbf{RR}^\top\mathbf{V} = \mathbf{PRR}^\top\mathbf{V} = \mathbf{PV}$.

Cost, in this order:

1. $\mathbf{CK}$ is $(k\times n)(n\times h)$, so nkh. The result is $k\times h$.
2. $\mathbf{DV}$ is $(k\times n)(n\times h)$, so nkh. The result is $k\times h$.
3. $\mathbf{Q(CK)}^\top$ is $(n\times h)(h\times k)$, so nkh. The result is $n\times k$.
4. $(\mathbf{Q(CK)}^\top)(\mathbf{DV})$ is $(n\times k)(k\times h)$, so nkh. The result is $n\times h$.

$$\text{Total} = 4nkh = O(nkh).$$

The order matters here too. Computing $(\mathbf{CK})^\top(\mathbf{DV})$ first gives an $h\times h$ matrix, and multiplying $\mathbf{Q}$ by it costs nh^2, which is more than nkh whenever $k < h$.

What it means. $\mathbf{C}$ and $\mathbf{D}$ squeeze the n keys and n values down to k rows each, so the attention matrix $\mathbf{Q(CK)}^\top$ is $n\times k$ instead of $n\times n$. This is the Linformer construction (Wang et al., 2020). Linformer learns its projections $\mathbf{E}, \mathbf{F} \in \mathbb{R}^{k\times n}$ in training instead of taking them from an SVD, because getting the SVD needs $\mathbf{P}$, which costs $O(n^2h)$ to build. Without the softmax, $\mathbf{QK}^\top$ is $(n\times h)(h\times n)$, so its rank is at most h anyway.

Check: with $n = 300$, $h = 64$, and $\mathbf{P}$ of rank 12, $\mathbf{Q(CK)}^\top(\mathbf{DV})$ matches $\mathbf{PV}$ with relative error 1.1×10^{-15}, with and without the scaling (`verification/verify_theory.py`).

5 Programming

In this programming homework, we will

- implement our own GPT - Transformer-based language model.
- learn about how to run Slurm jobs to train and evaluate the GPT LM on GPUs.

Skeleton Code and Structure:

The code base for this homework can be found at this GitHub repo under the `hw5` directory. Your task is to fill in the missing parts in the skeleton code, following the requirements, guidance, and tips provided in this pdf and the comments in the corresponding .py files. The code base has the following structure:

- `gpt` defines the Python module of the GPT LM
  - `bpe.py` creates a Byte-Pair Encoding tokenizer from scratch.
  - `model.py` implements our GPT model architecture from scratch.
  - `trainer.py` is the script for training, evaluating the GPT model, and visualizing the results.
  - `utils.py` defines helper functions.
- `main.py` provides the entry point to run your implementations of `gpt` module.
- `sbatch.sh` provides the bash script for submitting jobs to Slurm (details in 5.1.9)
- `hw5.md` provides instructions on how to setup the environment and run each part of the homework in `main.py`.
- `generate.py` provides helper functions for sampling from GPT LMs.
- `data.py` convert WikiText Data into LM training data.

TODOs — Your tasks include 1) generate plots and/or write short answers based on the results of running the code; 2) fill in the blanks in the skeleton to complete the code. We will explicitly mark these plotting, written answer, and filling-in-the-blank tasks as **TODOs** in the following descriptions, as well as a `# TODO` at the corresponding blank in the code.

Submission:

Your submission should contain two parts: 1) plots and short answers under the corresponding questions below; and 2) your completion of the skeleton code base, in a `.zip` file

> Where this ran: my laptop, not Slurm or Colab. The machine has Windows 11 Pro, an Intel Core Ultra 9 185H (16 cores), and an NVIDIA GeForce RTX 4050 Laptop GPU (6 GB). The software was Python 3.11.9, torch 2.1.0+cu121, transformers 4.37.1, datasets 2.17.1, and

matplotlib 3.8.3. The completed code is in the repository and in `Assignment5_code.zip` at its root.

Two changes outside the TODOs made it run there:

1. The trainer splits each batch of 256 into two chunks of 128 for the forward and backward pass (`micro_batches = 2` in `main.py`). Each chunk's loss is weighted by its share of the batch, so the summed gradient is the full-batch gradient. `verification/verify_microbatch.py` checks this to within 3×10^{-17}. It only lowers peak GPU memory, and it was on for both the CPU and the GPU runs. Without it, one batch of 256 peaks at 6.62 GB, more than the GPU's 6.44 GB. A step then took 4,667 ms instead of 270 ms (`verification/gpu_memory_probe.py`).
2. `scripts/run_main_local.py` runs `main.py` as written, with two differences. It reads WikiText-103 from local copies of the Hugging Face parquet files, and it saves the plots with matplotlib's Agg backend. Tk is broken in this Python install. The first full run trained to the end and then crashed in `plot()` on Tk, so the second run is the one reported here. Both reached the same validation loss, 4.83354, at step 20,000.

5.1 A GPT Language Model from Scratch

5.1.1 Introduction: Self-attention and Transformer

What do BERT, RoBERTa, GPT4, ... all have in common? *Self-attention and Transformer-based architecture*! Transformer-based architectures, which are primarily used in modeling language understanding tasks, eschew the use of recurrence in neural networks (RNNs) and instead trust entirely on self-attention mechanisms to draw global dependencies between inputs and outputs.

Some useful documentation:

As the architecture is so popular, there already exists a Pytorch module for `nn.Transformer` and a tutorial on how to use it for the next token prediction. However, we will implement it here ourselves, to get through to the smallest details.

There are of course many more tutorials out there about attention and Transformers. Below, we list a few that are worth exploring if you are interested in the topic and might want yet another perspective on the topic after this one:

- Transformer: A Novel Neural Network Architecture for Language Understanding (Jakob Uszkoreit, 2017) – The original Google blog post about the Transformer paper, focusing on the application in machine translation.
- The Illustrated Transformer (Jay Alammar, 2018) – A very popular and great blog post intuitively explaining the Transformer architecture with many nice visualizations.
- Attention? Attention! (Lilian Weng, 2018) – A nice blog post summarizing attention mechanisms in many domains including vision.

- Illustrated: Self-Attention (Raimi Karim, 2019) – A nice visualization of the steps of self-attention. Recommended going through if the explanation below is too abstract for you.
- The Transformer family (Lilian Weng, 2020) – A very detailed blog post reviewing more variants of Transformers besides the original one.

5.1.2 What is Attention?

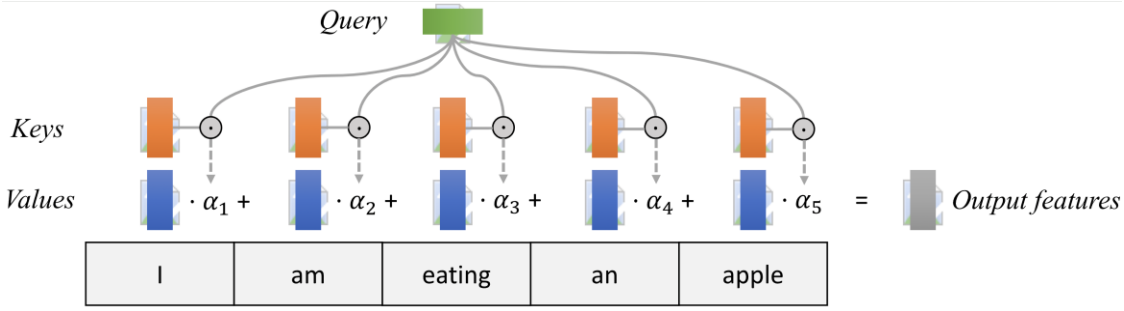
The attention mechanism describes a recent new group of layers in neural networks that has attracted a lot of interest in the past few years, especially in sequence tasks. There are a lot of different possible definitions of “attention" in the literature, but the one we will use here is the following: *the attention mechanism describes a weighted average of (sequence) elements with the weights dynamically computed based on an input query and elements’ keys*. So what does this exactly mean? The goal is to take an average of the features of multiple elements. However, instead of weighting each element equally, we want to weight them depending on their actual values. In other words, we want to dynamically decide on which inputs we want to”attend" more than others. In particular, an attention mechanism has usually four parts we need to specify:

- **Query**: The query is a feature vector that describes what we are looking for in the sequence, i.e. what would we maybe want to pay attention to.
- **Keys**: For each input element, we have a key which is again a feature vector. This feature vector roughly describes what the element is "offering", or when it might be important. The keys should be designed such that we can identify the elements we want to pay attention to based on the query.
- **Values**: For each input element, we also have a value vector. This feature vector is the one we want to average over.
- **Score function**: To rate which elements we want to pay attention to, we need to specify a score function f_{attn}. The score function takes the query and a key as input, and outputs the score/attention weight of the query-key pair. It is usually implemented by simple similarity metrics like a dot product.

The weights of the average are calculated by a Softmax overall score function outputs. Hence, we assign those value vectors a higher weight whose corresponding key is most similar to the query. If we try to describe it with pseudo-math, we can write:

$$\alpha_i = \frac{\exp\left(f_{attn}(\text{key}_i, \text{query})\right)}{\sum_j \exp\left(f_{attn}\left(\text{key}_j, \text{query}\right)\right)}, \qquad \text{out} = \sum_i \alpha_i \cdot \text{value}_i$$

Visually, we can show the attention over a sequence of words as follows. For every word, we have one key and one value vector. The query is compared to all keys with a score function (in this case the dot product) to determine the weights. The Softmax is not visualized for simplicity. Finally, the value vectors of all words are averaged using the attention weights.



Most attention mechanisms differ in terms of what queries they use, how the key and value vectors are defined, and what score function is used. The attention applied inside the Transformer architecture is called **self-attention**. In self-attention, each sequence element provides a key, value, and query. For each element, we perform an attention layer where based on its query, we check the similarity of all sequence elements' keys and returned a different, averaged value vector for each element. We will now go into a bit more detail by first looking at the specific implementation of the attention mechanism which is in the Transformer case the scaled dot product attention.

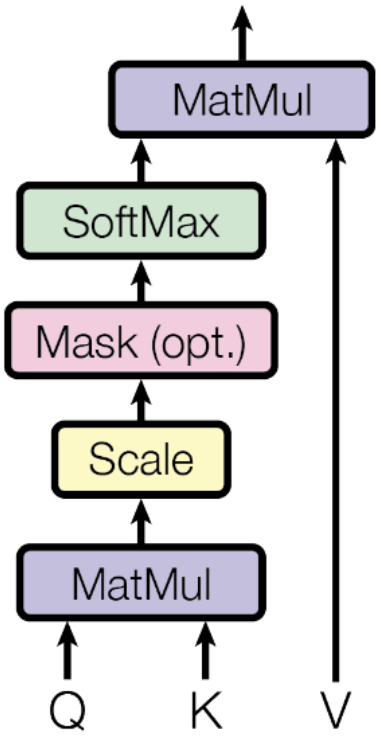
5.1.3 Scaled Dot Product Attention

The core concept behind self-attention is the scaled dot product attention. Our goal is to have an attention mechanism with which any element in a sequence can attend to any other while still being efficient to compute. The dot product attention takes as input a set of queries $Q \in \mathbb{R}^{T \times d_k}$, keys $K \in \mathbb{R}^{T \times d_k}$ and values $V \in \mathbb{R}^{T \times d_v}$ where T is the sequence length, and d_k and d_v are the hidden dimension for queries/keys and values respectively. For simplicity, we neglect the batch dimension for now. The attention value from element i to j is based on its similarity of the query Q_i and key K_j, using the dot product as the similarity metric. In math, we calculate the dot product attention as follows:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

The matrix multiplication QK^T performs the dot product for every possible pair of queries and keys, resulting in a matrix of the shape $T \times T$. Each row represents the attention logits for a specific element i to all other elements in the sequence. On these, we apply a Softmax and multiply with the value vector to obtain a weighted mean (the weights being determined by the attention). Another perspective on this attention mechanism offers the computation graph which is visualized below (figure credit:). There is also a nice visualization of these values are computed here.

Scaled Dot-Product Attention



One aspect we haven't discussed yet is the scaling factor of $1/\sqrt{d_k}$. This scaling factor is crucial to maintain an appropriate variance of attention values after initialization. Remember that we initialize our layers to have equal variance throughout the model, and hence, Q and K might also have a variance close to 1. However, performing a dot product over two vectors with a variance σ results in a scalar having d_k-times higher variance:

$$q_i \sim \mathcal{N}(0,\sigma), k_i \sim \mathcal{N}(0,\sigma) \rightarrow \text{Var}\left(\sum_{i=1}^{d_k} q_i \cdot k_i\right) = \sigma \cdot d_k$$

If we do not scale down the variance back to σ, the softmax over the logits will already saturate to 1 for one random element and 0 for all others. The gradients through the softmax will be close to zero so we can't learn the parameters appropriately.

5.1.4 Multi-Head Attention

The scaled dot product attention allows a network to attend over a sequence. However, often there are multiple different aspects a sequence element wants to attend to, and a single weighted average is not a good option for it. This is why we extend the attention mechanisms to multiple heads, i.e. multiple different query-key-value triplets on the same features. Specifically, given a query, key, and value matrix, we transform those into h sub-queries, sub-

keys, and sub-values, which we pass through the scaled dot product attention independently. Afterward, we concatenate the heads and combine them with a final weight matrix. Mathematically, we can express this operation as:

$$\begin{aligned} \text{Multihead}(Q, K, V) &= \text{Concat}(\text{head}_1, \ldots, \text{head}_h) W^O \\ \text{where head}_i &= \text{Attention}\left(QW_i^Q, KW_i^K, VW_i^V\right) \end{aligned}$$

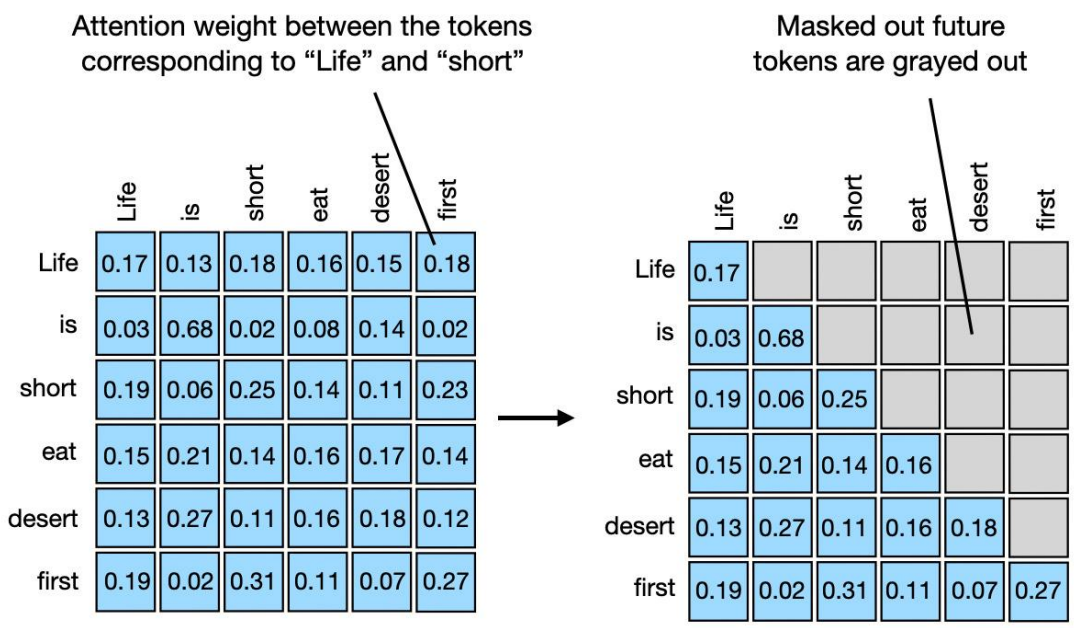
We refer to this as Multi-Head Attention layer with the learnable parameters $W_{1\ldots h}^Q \in \mathbb{R}^{D \times d_k}$, $W_{1\ldots h}^K \in \mathbb{R}^{D \times d_k}$, $W_{1\ldots h}^V \in \mathbb{R}^{D \times d_v}$, and $W^O \in \mathbb{R}^{h \cdot d_k \times d_{out}}$ (D being the input dimension). Expressed in a computational graph, we can visualize it below (figure credit:).

5.1.5 Causal Attention Masking

For GPT-like LMs, we train the model to generate one token at a time from left to right (autoregressive). When predicting the next token, the autoregressive setup constraints the model to only "see", i.e. attend to, the preceding tokens on the left-hand side. For instance, given a sample training text: "Life is short eat dessert first", the self-attention should be applied only within the tokens on the left-hand side of the →, in order to predict the token on the right-hand side:

- "Life" → "is"
- "Life is" → "short"
- "Life is short" → "eat"
- "Life is short eat" → "dessert"
- "Life is short eat dessert" → "first"

To achieve this in practice, one straightforward approach is to mask out future tokens for each timestamp by applying a mask to the attention weight matrix (causal attention masking). For the example above, we can apply the mask:



TODOs: Read and complete the missing lines in the `__init__` function of the `CausalSelfAttention` class in `gpt/model.py`, create the causal attention mask.
Hint: Follow the requirements and hints specified in the code comments.

Done in `Assignment5/gpt/model.py`:

```
causal_mask = torch.tril(torch.ones(config.block_size, config.block_size))
casual_mask = causal_mask   # the skeleton's next lines use this spelling
casual_mask = casual_mask.view(1, 1, config.block_size, config.block_size)
self.register_buffer("causal_mask", casual_mask)   # provided line
```

`torch.tril` keeps the diagonal and everything below it, so row t has 1s at keys 0 to t and 0s after. The skeleton's `register_buffer` line reads a variable spelled `casual_mask`, so I point that name at the same tensor. `verification/verify_attention.py` confirms the buffer is (1, 1, 32, 32) and lower triangular.

5.1.6 Putting it Together (so far): Multi-Head Self Attention with Causal Masking

Now that we have delved into the details of all important components, it's time to build the causal self-attention block for our own GPT!
TODOs: Read and complete the missing lines in the `forward` function of the `CausalSelfAttention` class in `gpt/model.py`.
Hint: Follow the requirements and hints specified in the code comments.

Done in `Assignment5/gpt/model.py`. The core of `CausalSelfAttention.forward`:

```
q, k, v = self.c_attn(x).split(self.n_embd, dim=2)
k = k.view(B, T, self.n_head, C // self.n_head).transpose(1, 2)
q = q.view(B, T, self.n_head, C // self.n_head).transpose(1, 2)
v = v.view(B, T, self.n_head, C // self.n_head).transpose(1, 2)
att = (q @ k.transpose(-2, -1)) / math.sqrt(k.size(-1))
```

```
att = att.masked_fill(self.causal_mask[:, :, :T, :T] == 0, float('-inf'))
att = F.softmax(att, dim=-1)
att = self.attn_dropout(att)
y = att @ v
y = y.transpose(1, 2).contiguous().view(B, T, C)    # provided line
y = self.resid_dropout(self.c_proj(y))
```

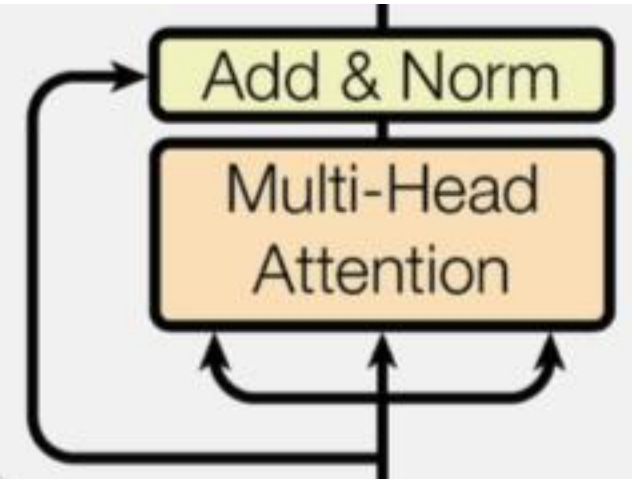
The scale is $1/\sqrt{d_k}$ with d_k = 16, the per-head size (48 split over 3 heads). The mask is cut to the actual length T, so batches shorter than 32 work.

Checks in `verification/verify_attention.py`, with dropout off:

1. The output matches PyTorch's `scaled_dot_product_attention(is_causal=True)` to within 1.2×10^{-7}.
2. Changing the tokens after position 9 leaves the outputs at positions 0 to 9 unchanged.
3. Every attention row sums to 1, with zero weight on later positions.
4. The full GPT learns a simple next-token rule on synthetic data. Its loss went from 3.921 (about $\ln 50 = 3.912$, a uniform guess) to 0.004 in 300 steps.

5.1.7 Residual connection and normalization

Notice that there is a residual connection and normalization on top of the multi-head attention layer.



Taking as input x, it is first passed through a Multi-Head Attention block as we have implemented above. The output is added to the original input using a residual connection, and we apply a consecutive Layer Normalization on the sum. Overall, it calculates LayerNorm$\big(x + \text{Multihead}(x, x, x)\big)$ (x being Q, K and V input to the attention layer). The residual connection is crucial in the Transformer architecture for two reasons:

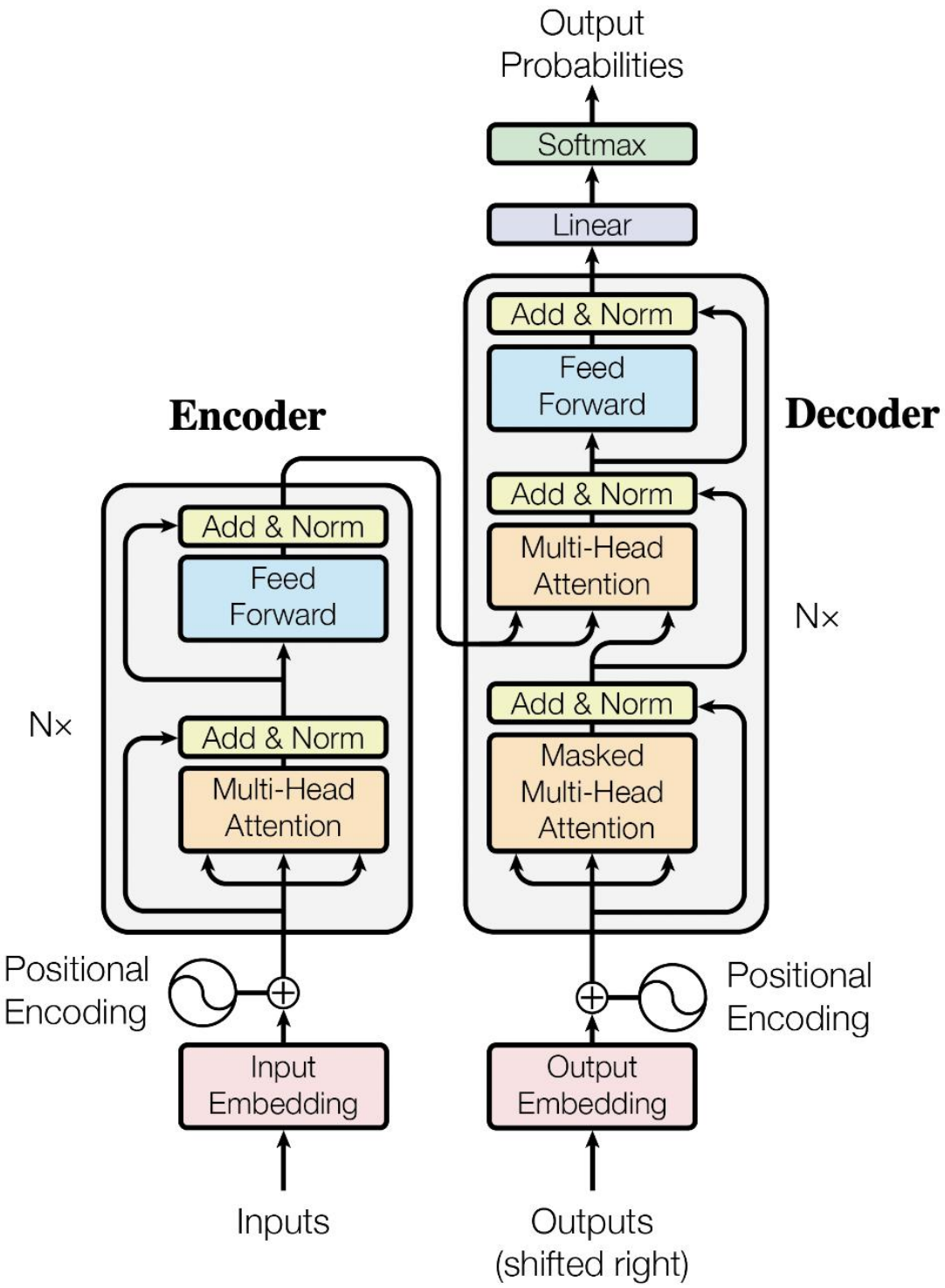
1. Similar to ResNets, Transformers are designed to be very deep. Some models contain more than 24 blocks in the encoder. Hence, the residual connections are crucial for enabling a smooth gradient flow through the model.

2. Without the residual connection, the information about the original sequence is lost. Remember that the Multi-Head Attention layer ignores the position of elements in a sequence, and can only learn it based on the input features. Removing the residual connections would mean that this information is lost after the first attention layer (after initialization), and with a randomly initialized query and key vector, the output vectors for position i have no relation to its original input. All outputs of the attention are likely to represent similar/same information, and there is no chance for the model to distinguish which information came from which input element. An alternative option to residual connection would be to fix at least one head to focus on its original input, but this is very inefficient and does not have the benefit of the improved gradient flow.

The Layer Normalization also plays an important role in the Transformer architecture as it enables faster training and provides small regularization. Additionally, it ensures that the features are in a similar magnitude among the elements in the sequence. We are not using Batch Normalization because it depends on the batch size which is often small with Transformers (they require a lot of GPU memory), and BatchNorm has shown to perform particularly badly in language as the features of words tend to have a much higher variance (there are many, very rare words which need to be considered for a good distribution estimate).

5.1.8 Putting it Together: Transformer Architecture

Originally, the Transformer model was designed for machine translation. Hence, it has an encoder-decoder structure where the encoder takes as input the sentence in the original language and generates an attention-based representation. *This is different from our GPT implementation, which is a decoder-only architecture.* On the other hand, the decoder attends to the encoded information and generates the translated sentence in an auto-regressive manner, as in a standard RNN. The full Transformer architecture looks as follows (figure credit:):



The encoder consists of N identical blocks. These blocks contain Self-Attention, Layer Normalization, and Residual connections. Additionally, a small fully connected feed-forward network is added to the model, which is applied to each position separately and identically. Specifically, the model uses a Linear→ReLU→Linear MLP. The full transformation including the residual connection can be expressed as:

$$\begin{aligned} \text{FFN}(x) &= max(0, xW_1 + b_1)W_2 + b_2 \\ x &= \text{LayerNorm}\big(x + \text{FFN}(x)\big) \end{aligned}$$

This MLP adds extra complexity to the model and allows transformations on each sequence element separately. You can imagine as this allows the model to "post-process" the new information added by the previous Multi-Head Attention, and prepare it for the next attention block. Usually, the inner dimension of the MLP is 2-8× larger than d_{model}, i.e. the dimension of the original input x. The general advantage of a wider layer instead of a narrow, multi-layer MLP is the faster, parallelizable execution.

5.1.9 Train and Evaluate GPT LM on GPUs

Now let's train our GPT LM!

From this homework on, we will be using a freely-available GPU, such as through Google Colab.

Why GPUs? A crucial feature of PyTorch is the support of GPUs, short for Graphics Processing Unit. A GPU can perform many thousands of small operations in parallel, making it very well suitable for performing large matrix operations in neural networks. When comparing GPUs to CPUs, we can list the following main differences:

CPUs and GPUs have both different advantages and disadvantages, which is why many computers contain both components and use them for different tasks. In case you are not familiar with GPUs, you can read up more details in this NVIDIA blog post or here.

GPUs can accelerate the training of your network up to a factor of 100 which is essential for large neural networks. PyTorch implements a lot of functionality to support GPUs (mostly those of NVIDIA due to the libraries CUDA and cuDNN).

To get a more direct comparison, let's run our model for 50 training steps with both CPU and GPU.
TODOs: Run the `cpu_gpu_comparison` in `main.py`, and fill in the running time below (will be printed out on the console):

> Answer, for 50 training steps (batch 256, block size 32, gpt-nano with 4,911,120 parameters):

Device	Running time
CPU (Intel Core Ultra 9 185H)	156.88 s
GPU (NVIDIA RTX 4050 Laptop)	10.65 s

> That's 14.7 times faster on the GPU. These are the times `main.py` printed in the run that made the plots below (`Assignment5/run_log.txt`). The timer covers the 50 steps only, not the data loading or the validation pass before step 0. (`main.py` prints 2.50M parameters because its count leaves out the 2,412,336 in the output layer.)

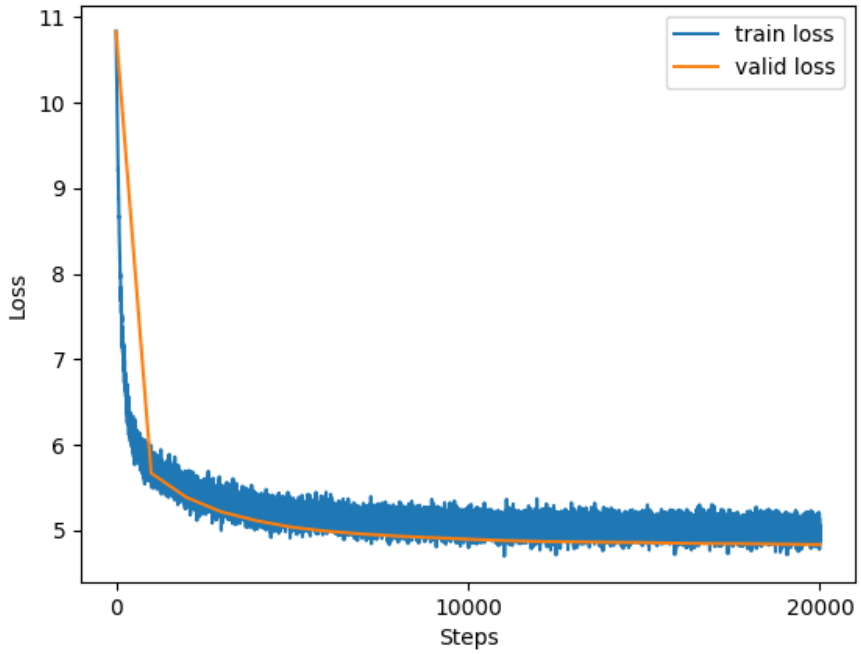
The CPU time wasn't stable between runs, so I measured the same comparison three more times (`verification/cpu_gpu_repeat.py`, which calls `main.run` with the same settings):

Run	CPU (s)	GPU (s)	CPU / GPU
Sep 29, first attempt (`Assignment5/run_log_first_attempt.txt`)	107.50	10.69	10.1
Sep 29, reported run (`Assignment5/run_log.txt`)	156.88	10.65	14.7
Oct 19, repeat 1	117.41	13.98	8.4
Oct 19, repeat 2	107.36	13.77	7.8
Oct 19, repeat 3	114.94	13.80	8.3

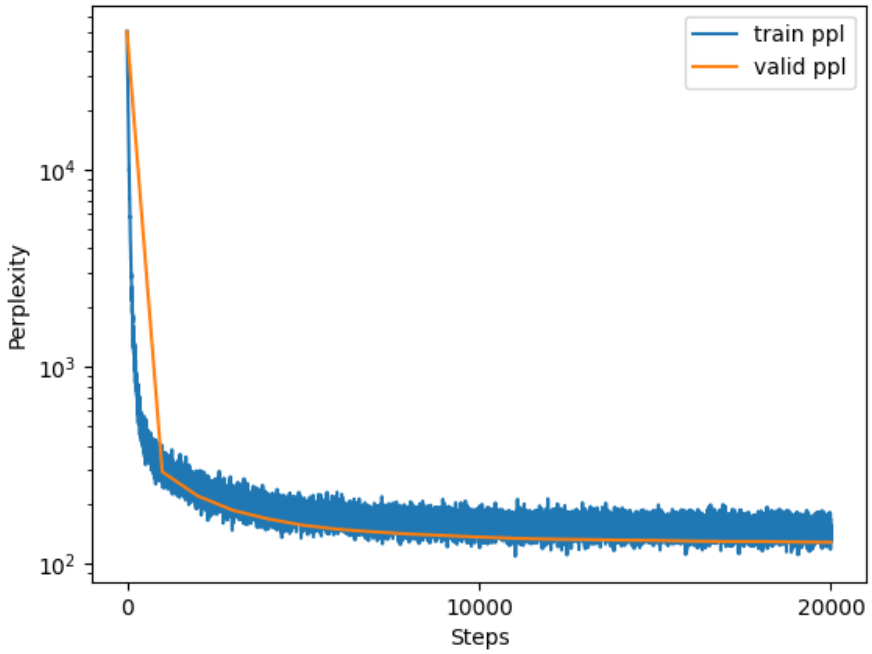
Leaving out the 156.88 s run, the CPU stayed between 107 and 118 s, so that run looks like an outlier. A likely cause is the chip itself. PyTorch ran 16 CPU threads, one per core. The 185H's 16 cores are a mix of fast and slow ones (6 performance, 8 efficient, 2 low-power), so where Windows puts the threads can change the time. I didn't measure that, though. The GPU time was steady within each day but about 30% slower on October 19. The laptop was plugged in and on the Balanced power plan, and I don't have a measured reason for that either. Across all five runs the GPU was 7.8 to 14.7 times faster, with a median of 8.4.

Finally, let's run the full training and evaluation on GPU.
TODOs: Run the `gpu_full_run` function in `main.py` to train the GPT language model, paste the two generated plots here. This function will also sample from the trained GPT LM. paste the completion to the prefix after the plots.

Answer: both plots are below. They come from the 20,000-step GPU run, which took 4,254.87 s (about 71 minutes). Validation runs every 1,000 steps.



Training loss at every step and validation loss every 1,000 steps (`Assignment5/loss.png`, as `gpu_full_run` saved it).



Training and validation perplexity on a log scale (`Assignment5/ppl.png`, as `gpu_full_run` saved it).

What the plots show. At step 0 the validation loss is 10.83, about $\ln 50{,}257 = 10.82$, the loss of a uniform guess over the vocabulary. It drops fast, to 5.67 (perplexity 294.05) by step 1,000, and then slowly. At step 20,000 it's 4.834, perplexity 128.96. Validation improved at all 20 checks after step 0, so the saved `model.pth` is the step-20,000 model. It was still improving a little at the end.

The per-step training loss is noisy because each point is one batch. Over the last 1,000 steps it averaged 5.01, with 90% of steps between 4.87 and 5.15. That's a bit above validation, probably because dropout (0.1) is on in training and off in evaluation. There's no sign of overfitting, which fits a model this small on WikiText-103 (4.9M parameters). One more detail: the trainer's validation perplexity is the average of each batch's perplexity, so it sits a little above $\exp(4.834) = 125.7$.

Completions. `sample_from_trained_model` loads `model.pth` and samples 30 new tokens for each of four prompts. At each step it picks from the 40 most likely tokens (seed 3407, set in `generate.py`). The prompts were these four, each starting with a space in `main.py`:

1. The best perks of living on the east
2. It is a truth universally acknowledged
3. The best way to learn
4. According to the latest research

The console output, verbatim, with each prompt followed by its completion. I left out the 80-dash separator line `main.py` prints before each sample, since it wraps on the page.

```
 The best perks of living on the east of the United States under the island ,
it moved a time of the river to the westbound lanes and north side .
 as the highway route

 It is a truth universally acknowledged that we had a child , a character and
a child . "
 that may only come out a lot for some of her life 's death

 The best way to learn to prevent the rest of the season ( one ) by Mascoy ,
later on the second side of the first time , a " This game .

 According to the latest research in the English Civil War to its official
public economy , its name was published as part of the National Academy of
the country , or the original Greek Republic
```

The model picked up WikiText's style, including the spaces around punctuation, and Wikipedia topics like highways and the English Civil War. Short phrases are mostly grammatical, but it loses the thread within a sentence. That's about what I'd expect from 3 layers, 48-dimensional states, and a 32-token context.